IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

The Yenepoya institute of arts , science, commerce and management

Project – Low Level Design

on

AI-Enhanced CI/CD Pipeline for EdTech Software

Student Name: Keerthi

Industry Mentor: Mr. Shashank **Guided by: Ms. Manjula**

Table of Contents

1. Introduction 3

1.1. Scope of the Document 3

1.2. Intended Audience 3

1.3. System Overview 3

2. Module Specifications 4

2.1. Pipeline Orchestrator 4

2.2. Build Stage 4

2.3. Test Stage 4

2.4. AI Analysis Stage 5

3. EdTech Application Module 7

4. Test Specifications 9

5. File Structure and Organization 10

6. Error Handling Strategy 10

7. Logging Specifications 10

8. GitHub Actions Integration 11

9. Performance Optimization 11

10. Security Implementation 11

11. References 12

1. Introduction

This Low-Level Design document provides detailed implementation specifications for the AI-Enhanced CI/CD Pipeline system. It describes the internal structure of each module, detailed algorithms, data structures, function signatures, error handling mechanisms, and implementation guidelines. This document serves as a blueprint for developers implementing the system and maintains traceability to the High-Level Design.

1.1 Scope of the Document

This document covers detailed module specifications including class definitions with attributes and methods, function-level implementation details with pseudocode, data structure specifications, algorithm descriptions, file organization and naming conventions, error handling strategies, logging specifications, and detailed test specifications for each module.

1.2 Intended Audience

Software developers implementing the pipeline modules, quality assurance engineers writing test cases, code reviewers validating implementation correctness, and maintenance teams understanding system internals for debugging and enhancement purposes.

1.3 System Overview

The system consists of five primary Python modules organized in a pipeline directory structure. The main orchestrator coordinates sequential execution of build, test, analysis, and deployment stages. Each module is self-contained with minimal coupling to other modules, communicating through standardized return value conventions.

2. Module Specifications

2.1 Pipeline Orchestrator Module (run_pipeline.py)

Purpose: Coordinates execution of all pipeline stages in sequential order and manages overall pipeline state. Main Function - main() with Parameters: None, Returns: int (exit code 0 for success, 1 for failure). Algorithm: Import all stage modules. Print pipeline start banner. Initialize stages list containing tuples of stage name and function reference. For each stage in stages list: print stage start message, call stage function and capture result tuple, if result success is False then print failure message and return exit code 1, print stage success message. Print pipeline completion banner. Return exit code 0. Dependencies: sys module for exit codes, pipeline.build_stage, pipeline.test_stage, pipeline.ai_analysis_stage, pipeline.deploy_stage. Error Handling: Catches ImportError if stage modules not found. Catches any exception during stage execution and logs with traceback. Logging: Prints formatted console output with stage names and status indicators using print statements with visual separators.

2.2 Build Stage Module (pipeline/build_stage.py)

Purpose: Validates build environment and verifies presence of required files. Main Function - run_build_checks() with Parameters: None, Returns: tuple of (bool success, str message). Algorithm: Import sys and os modules. Check Python version using sys.version_info, if version less than 3.6 return False with error message. Define required_files list containing paths to pipeline modules and sample code. For each file in required_files: use os.path.exists to check file presence, if file not found return False with missing file message. Return True with success message if all checks pass. Helper Function - check_python_version() with Parameters: None, Returns: bool. Checks if sys.version_info.major equals 3 and minor greater than or equal to 6. Helper Function - verify_file_structure() with Parameters: None, Returns: tuple of (bool, list of missing files). Iterates through required files and builds list of missing files. Dependencies: sys module for version info, os.path module for file existence checks. Error Handling: Returns False tuple on any validation failure with descriptive message. Does not raise exceptions. Logging: Prints Python version information and file check results to console.

2.3 Test Stage Module (pipeline/test_stage.py)

Purpose: Discovers and executes all unit tests using Python unittest framework. Main Function - run_tests() with Parameters: None, Returns: tuple of (bool success, str message). Algorithm: Import unittest and sys modules. Create TestLoader instance. Discover tests in tests directory using loader.discover with start_dir equals tests and pattern equals test underscore asterisk dot py. Create TextTestRunner with verbosity level 2. Run test suite using runner.run and capture result. Check result.wasSuccessful(), return True with pass message if successful else return False with failure message including failure count. Dependencies: unittest module for test

framework, sys module for system operations. Error Handling: Catches exceptions during test discovery or execution and returns False with error details. Wraps test execution in try-except block. Logging: Test runner automatically prints detailed test execution results to console including test names, status, and failure details.

2.4 AI Analysis Stage Module (pipeline/ai_analysis_stage.py)

Purpose: Performs static code analysis using Abstract Syntax Tree parsing and rule-based quality checks. Main Function - run_ai_analysis() with Parameters: None, Returns: tuple of (bool success, str message). Algorithm: Import ast, os modules. Get list of Python files in sample_code directory. Initialize empty issues list. For each file: call analyze_code function and append results to issues list. Call generate_report with issues list. Return True with analysis complete message. Helper Function - analyze_code(filepath) with Parameters: filepath (str), Returns: dict with keys file, issues. Algorithm: Read file content using open and read. Parse content using ast.parse to create syntax tree. Initialize empty issues list. Create NodeVisitor subclass to traverse tree. For each FunctionDef node: count lines in function body, if count greater than 15 append long function issue. Check if docstring exists using ast.get_docstring, if None append missing docstring issue. For each Name node in arguments: if name length less than 2 append short variable name issue. For each line in file: if length greater than 79 append line too long issue. If line contains TODO append TODO comment issue. Return dict with filepath and issues list. Helper Function - generate_report(issues) with Parameters: issues (list of dicts), Returns: bool. Algorithm: Create reports directory if not exists using os.makedirs. Open ai_analysis_report.txt in write mode. Write header and timestamp. For each file in issues: write filename header, for each issue write severity, type, line number, and recommendation. Close file and return True. Data Structures: Issue dictionary contains severity (str), type (str), line_number (int), message (str), recommendation (str). Dependencies: ast module for syntax tree parsing, os module for file operations, datetime for timestamps. Error Handling: Catches SyntaxError during AST parsing and logs as analysis issue. Catches IOError during report writing and returns False. Logging: Prints analysis progress and issue counts to console.

2.5 Deploy Stage Module (pipeline/deploy_stage.py)

Purpose: Simulates deployment process and creates deployment artifacts. Main Function - run_deployment() with Parameters: None, Returns: tuple of (bool success, str message). Algorithm: Import datetime and os modules. Create reports directory if not exists. Generate deployment timestamp using datetime.now. Create deployment log filename with timestamp. Call create_deployment_log function with log filename. Print deployment simulation messages. Return True with deployment success message. Helper Function - create_deployment_log(filename) with Parameters: filename (str), Returns: bool. Algorithm: Open file in write mode. Write deployment header with timestamp. Write deployment metadata

including version, environment, status. Write deployment steps log. Close file and return True. Dependencies: datetime for timestamps, os for file operations. Error Handling: Catches IOError during file creation and returns False with error message. Logging: Prints deployment progress to console and writes detailed log to file.

3. EdTech Application Module Specifications

3.1 Student Class (sample_code/edtech_app.py)

Purpose: Represents a student entity with course enrollment capabilities. Attributes: student_id (str) unique identifier, name (str) student full name, email (str) student email address, enrolled_courses (list) list of course IDs. Constructor Method - __init__(self, student_id, name, email) with Parameters: student_id (str), name (str), email (str). Initializes all attributes and sets enrolled_courses to empty list. Method - add_course(self, course_id) with Parameters: course_id (str), Returns: bool. Algorithm: Check if course_id already in enrolled_courses, if yes return False. Append course_id to enrolled_courses and return True. Method - remove_course(self, course_id) with Parameters: course_id (str), Returns: bool. Algorithm: Check if course_id in enrolled_courses, if yes remove it and return True, else return False. Method - get_courses(self) with Parameters: None, Returns: list of str. Returns copy of enrolled_courses list. Method - get_student_info(self) with Parameters: None, Returns: dict. Returns dictionary with student_id, name, email, and course count.

3.2 Course Class

Purpose: Manages course information and student enrollment. Attributes: course_id (str), title (str), instructor (str), max_students (int), enrolled_students (list). Constructor Method - __init__(self, course_id, title, instructor, max_students) initializes attributes with enrolled_students as empty list. Method - enroll_student(self, student_id) with Parameters: student_id (str), Returns: bool. Algorithm: Check if len of enrolled_students less than max_students, if no return False with message course full. Check if student_id already enrolled, if yes return False. Append student_id to enrolled_students and return True. Method - drop_student(self, student_id) with Parameters: student_id (str), Returns: bool. Removes student from enrolled_students if present. Method -

get_enrolled_students(self) with Parameters: None, Returns: list. Returns copy of enrolled_students list. Method - get_enrollment_count(self) with Parameters: None, Returns: int. Returns length of enrolled_students list. Method - is_full(self) with Parameters: None, Returns: bool. Returns True if enrollment count equals max_students.

3.3 Assessment Class

Purpose: Tracks student assessment scores and statistics. Attributes: assessment_id (str), course_id (str), title (str), max_score (int), scores (dict mapping student_id to score). Constructor Method - __init__(self, assessment_id, course_id, title, max_score) initializes attributes with scores as empty dictionary. Method - submit_score(self, student_id, score) with Parameters: student_id (str), score (int or float), Returns: bool. Algorithm: Validate score is between 0 and max_score, if invalid return False. Store score in scores dictionary with student_id as key. Return True. Method - get_score(self, student_id) with Parameters: student_id (str), Returns: int or float

or None. Returns score from scores dictionary if student_id exists else None. Method - get_all_scores(self) with Parameters: None, Returns: dict. Returns copy of scores dictionary. Method - calculate_average(self) with Parameters: None, Returns: float. Algorithm: If scores dictionary empty return 0.0. Calculate sum of all scores and divide by count. Return average rounded to 2 decimal places. Method - get_statistics(self) with Parameters: None, Returns: dict. Calculates and returns dictionary containing average, min, max, and submission count.

4. Test Specifications

4.1 Student Class Tests (tests/test_edtech_app.py)

Test Class - TestStudent extends unittest.TestCase. Test Method - test_student_creation verifies student object initialization with correct attributes. Test Method - test_add_course verifies course can be added to student enrollment. Uses assertEqual to check course appears in enrolled_courses list. Test Method - test_add_duplicate_course verifies duplicate course addition returns False. Test Method - test_remove_course verifies course removal functionality. Test Method - test_get_courses verifies get_courses returns correct list. Test Method - test_get_student_info verifies student info dictionary structure and content.

4.2 Course Class Tests

Test Class - TestCourse extends unittest.TestCase. Test Method - test_course_creation verifies course initialization. Test Method - test_enroll_student verifies student enrollment succeeds when capacity available. Test Method - test_enroll_full_course verifies enrollment fails when course at max capacity. Test Method - test_drop_student verifies student can be dropped from course. Test Method - test_get_enrollment_count verifies enrollment count calculation. Test Method - test_is_full verifies full course detection.

4.3 Assessment Class Tests

Test Class - TestAssessment extends unittest.TestCase. Test Method - test_assessment_creation verifies assessment object initialization. Test Method - test_submit_score verifies score submission within valid range. Test Method - test_submit_invalid_score verifies rejection of scores outside valid range. Test Method - test_get_score verifies score retrieval for enrolled student. Test Method - test_calculate_average verifies average score calculation. Test Method - test_get_statistics verifies statistical calculations including min, max, and average.

5. File Structure and Organization

Root Directory: Contains run_pipeline.py as main entry point. Pipeline Directory: Contains four stage modules - build_stage.py, test_stage.py, ai_analysis_stage.py, deploy_stage.py. Each module must include __init__ method marker for package recognition. Sample Code Directory: Contains edtech_app.py with Student, Course, and Assessment classes. Tests Directory: Contains test_edtech_app.py with all test cases. Must include __init__.py for test discovery. Reports Directory: Auto-created during pipeline execution. Contains ai_analysis_report.txt and deployment_log.txt. GitHub Workflows Directory: Contains pipeline.yml defining GitHub Actions workflow configuration. Naming Conventions: Use snake_case for file names and function names. Use PascalCase for class names. Prefix test files with test underscore. Use descriptive names indicating module purpose.

6. Error Handling Strategy

All pipeline stage functions return tuple of (bool, str) for consistent error propagation. Use try-except blocks to catch specific exceptions. ImportError for missing modules, IOError for file operations, SyntaxError for code parsing, ValueError for invalid data. Log exception details including traceback for debugging. Return False with descriptive error message on exception. Never use bare except clauses. Always specify exception types. Validate input parameters before processing. Check file existence before reading. Validate data types and ranges for numerical inputs. Use assertions for internal consistency checks during development.

7. Logging Specifications

Console Logging: Use print statements for real-time pipeline progress. Include visual separators for stage boundaries using equal signs or dashes. Display stage names with clear START and COMPLETE markers. Show test execution results with pass/fail indicators. File Logging: AI analysis reports written to reports/ai_analysis_report.txt with timestamp header. Deployment logs written to reports/deployment_log.txt with structured format. Include severity levels in analysis reports: LOW, MEDIUM, HIGH. Timestamp Format: Use ISO 8601 format YYYY-MM-DD HH:MM:SS for all timestamps. Log Content: Include operation being performed, result status, execution duration where applicable, error details with stack traces for failures. Avoid logging sensitive information such as credentials or personal data.

8. GitHub Actions Integration

Workflow File Location: .github/workflows/pipeline.yml. Workflow Name: CI/CD Pipeline. Trigger Events: Push to main branch, Pull request to main branch. Job Name: build. Runner: ubuntu-latest. Steps: Checkout code using actions/checkout@v2. Set up Python 3.9 using actions/setup-python@v2. Run pipeline using python run_pipeline.py. Upload reports as artifacts using actions/upload-artifact@v2. Artifact Configuration: Name pipeline-reports. Path reports directory. Retention days 30. Environment Variables: Can be defined in workflow for configuration. Secrets: Store sensitive data in GitHub repository secrets. Access via secrets.SECRET_NAME syntax.

9. Performance Optimization

Minimize file I/O operations by batching reads and writes. Use generator expressions for large data processing to reduce memory footprint. Parse AST trees once per file and cache results if multiple analyses needed. Implement early termination in loops when search criteria met. Use built-in Python functions and list comprehensions for performance. Avoid unnecessary object creation in tight loops. Profile code using cProfile module to identify bottlenecks during development. Target execution time under 30 seconds for typical codebase. Optimize test execution by grouping related tests and using setUp/tearDown efficiently.

10. Security Implementation

Input Validation: Validate all file paths to prevent directory traversal attacks using os.path.abspath and checking path starts with expected directory. Sanitize user inputs if accepting command line arguments. Code Execution: Use ast.parse for code analysis instead of exec or eval to prevent arbitrary code execution. Run pipeline in isolated environment without network access. Credential Management: Never hardcode credentials in source code. Use environment variables for sensitive configuration. Access GitHub secrets through proper GitHub Actions syntax. File Permissions: Set appropriate file permissions for generated reports. Restrict write access to reports directory. Dependency Security: Use only Python standard library to minimize attack surface. Regularly update Python runtime to latest stable version with security patches.

11. References

Python 3 Documentation - https://docs.python.org/3/. Python unittest Module Documentation. Python ast Module Documentation - Abstract Syntax Trees. PEP 8 Style Guide for Python Code. GitHub Actions Workflow Syntax Documentation. Python Testing Best Practices. Clean Code by Robert C. Martin. The Pragmatic Programmer by Andrew Hunt and David Thomas.